МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт искусственного интеллекта

Кафедра проблем управления

ПРАКТИЧЕСКАЯ РАБОТА №3

по дисциплине **Информационные элементы робототехнических систем**

Тема практической работы: «Аугментация данных»

Зенина А.А. _____

Студенты группы: КРБО-03-23 Грачев А.В. _____

Гришаев А.К. _____

Преподаватель: ст. преподаватель Бычков А.М. _____

Работа представлена к защите: «___» _____ 2026 г.

Москва 2026

1. Цель работы

Изучить основные методы аугментации данных для увеличения объёма обучающей выборки для повышения устойчивости нейросетевых моделей к вариативности входных данных. Создать подходящий для обучения датасет из нескольких оригинальных фото.

2. Задача работы

Реализовать пайплайн аугментации изображений с помощью библиотеки Albumentations, включающий следующие преобразования:

- геометрические (горизонтальное/вертикальное отражение, поворот, аффинные искажения);
- цветовые (изменение яркости/контрастности, HSV, гамма-коррекция);
- шумовые (гауссов шум, размытие);
- регуляризационные (CutOut);
- нелинейные деформации (оптическое, сеточное и эластичное искажения).

Применить пайплайн к исходны изображениям и получить аугментированный датасет.

3. Теоретические сведения

Аугментация данных – это метод искусственного расширения обучающей выборки путём применения различных преобразований к исходным изображениям. Это позволяет улучшить обобщающую способность модели, снизить переобучение и повысить устойчивость к изменениям условий съёмки.

В работе использована библиотека Albumentations – высокопроизводительная библиотека для аугментации изображений, поддерживающая множество типов преобразований и интеграцию с популярными фреймворками глубокого обучения.

Ниже приведены основные методы аугментации, включенные в пайплайн, с описанием их параметров и эффекта.

1. HorizontalFlip – Горизонтальное отражение

Отражает изображение зеркально относительно вертикальной оси. Применяется с вероятностью 0.5. Помогает модели научиться распознавать объекты независимо от их горизонтальной ориентации.

2. VerticalFlip – Вертикальное отражение

Отражает изображение относительно горизонтальной оси. Вероятность 0.3. Полезно для сцен, где объекты могут встречаться перевёрнутыми.

3. Rotate – Поворот

Поворачивает изображение на случайный угол в диапазоне [-30°, +30°] с вероятностью 0.7. Учит модель устойчивости к наклонам.

4. Affine – Аффинное преобразование

Комбинирует масштабирование (0.8–1.2), сдвиг (до 10%) и поворот (-15°…+15°) с вероятностью 0.5. Обеспечивает инвариантность к изменению масштаба и положения объекта.

5. RandomBrightnessContrast – Изменение яркости и контрастности

Изменяет яркость и контрастность в пределах ±20% с вероятностью 0.6. Имитирует различные условия освещения.

6. GaussNoise – Гауссов шум

Добавляет шум с дисперсией, эквивалентной значениям пикселей от 10 до 50. Вероятность 0.4. Повышает робастность к шуму сенсора.

7. GaussianBlur – Размытие по Гауссу

Сглаживает изображение ядром 3×3 или 7×7 с вероятностью 0.3. Имитирует расфокусировку.

8. CoarseDropout – Случайное вырезание прямоугольников (CutOut)

Заменяет от 1 до 8 прямоугольных областей размером 8–32 пикселя на черный цвет. Вероятность 0.3. Эффективная регуляризация.

9. ToGray – Преобразование в оттенки серого

С вероятностью 0.2 переводит изображение в ч/б. Учит модель не полагаться только на цвет.

10. HueSaturationValue – Изменение оттенка, насыщенности и яркости (HSV)

Смещает оттенок до 20°, насыщенность до 30%, яркость до 20% с вероятностью 0.5. Создаёт цветовое разнообразие.

11. OpticalDistortion – Оптическое искажение

Симулирует дисторсию объектива (бочка/подушка) интенсивностью до 5% с вероятностью 0.3. Повышает устойчивость к искажениям объектива.

12. GridDistortion – Сеточное искажение

Деформирует изображение путём смещения узлов сетки 5×5. Максимальное смещение 30% с вероятностью 0.3. Имитирует нежесткие деформации.

13. ElasticTransform – Эластичная деформация

Создаёт случайное поле смещений с параметрами alpha=1, sigma=50. Вероятность 0.2. Полезна для медицинских изображений и текстур.

14. RandomGamma – Коррекция гаммы

Нелинейно изменяет яркость (γ от 0.8 до 1.2) с вероятностью 0.4. Сохраняет локальный контраст.

15. CLAHE – Адаптивное выравнивание гистограммы

Улучшает локальную контрастность с clip_limit=2.0, сеткой 8×8, вероятность 0.3. Полезно для изображений с неравномерным освещением.

Все преобразования объединены в единый пайплайн с заданными вероятностями, что обеспечивает сбалансированное разнообразие данных.

4. Расчетно-графическая часть

Для демонстрации работы пайплайна аугментации было взято исходное изображение. К нему применены преобразования, описанные в теоретической части. На рисунках 1–4 представлены исходное и три аугментированных изображения.

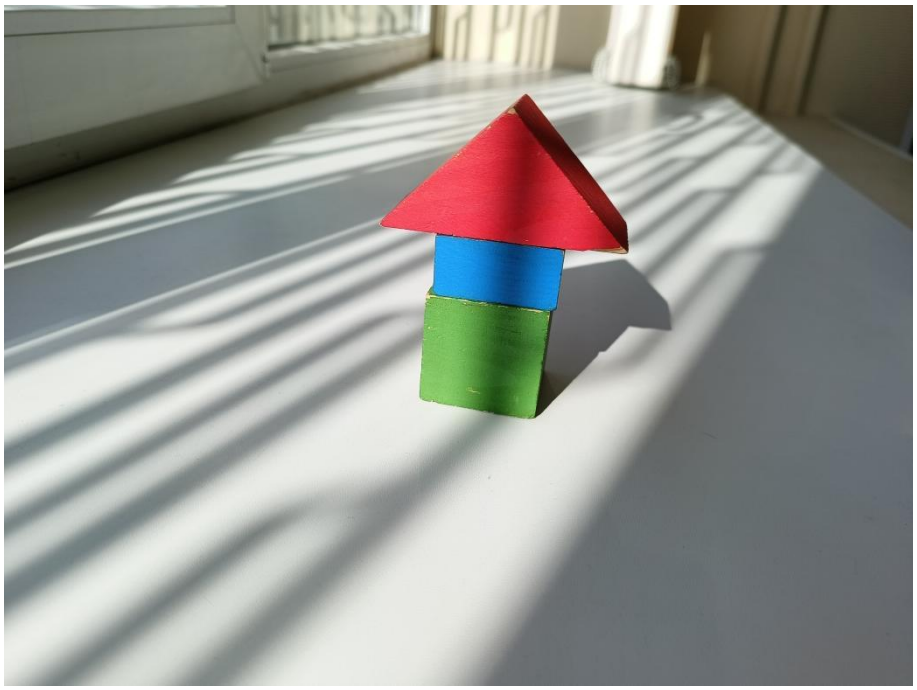

Рисунок 1 – Исходное изображение

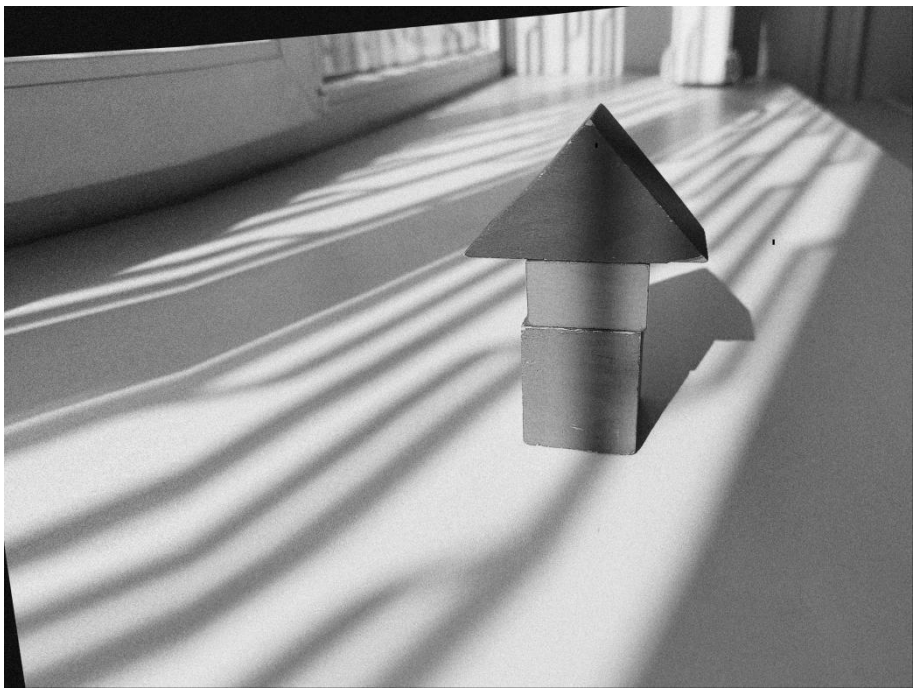

Рисунок 2 – Результат аугментации 1

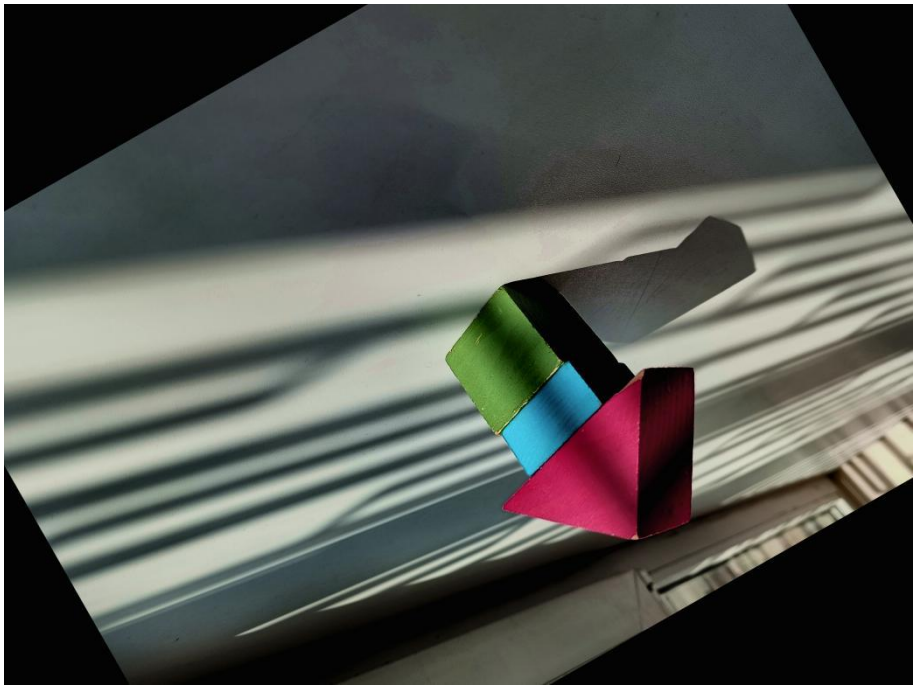

Рисунок 3 – Результат аугментации 2

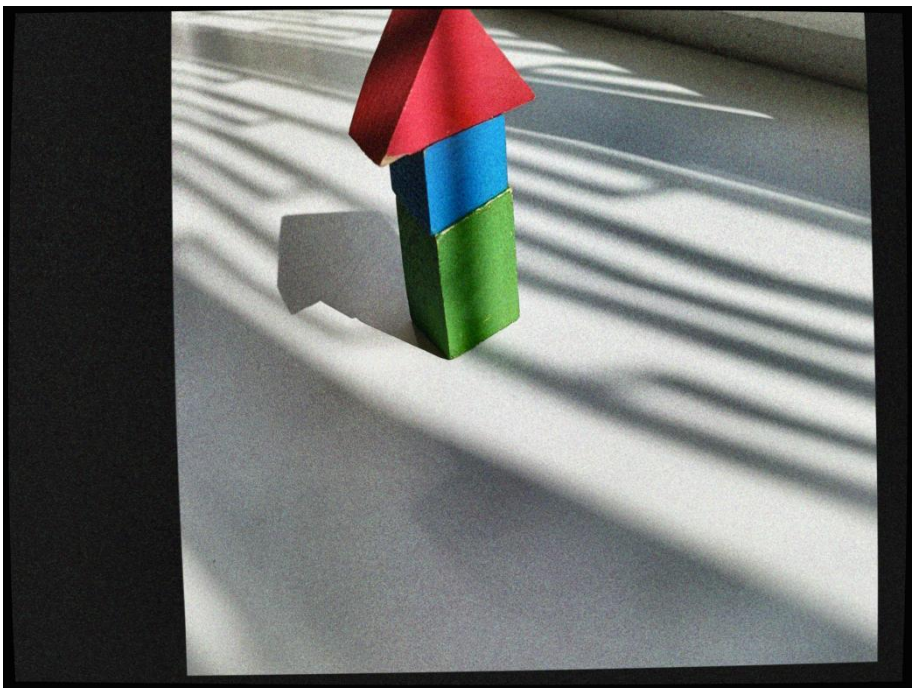

Рисунок 4 – Результат аугментации 3

Для аугментации были использованы 5 фото с объектом и 5 фото без объекта. Итоговый датасет содержит 200 фото.

Аугментация позволяет генерировать разнообразные варианты одного и того же объекта, что расширяет обучающую выборку и делает модель устойчивее к возможным искажениям в реальных условиях. Каждый из методов вносит специфические изменения: геометрические преобразования меняют ракурс, цветовые – имитируют смену освещения, а шумовые – добавляют артефакты съёмки. Регуляризационные методы (CutOut) заставляют модель игнорировать отдельные участки изображения, что предотвращает запоминание шумовых паттернов.

5. Выводы по работе

В ходе работы был реализован пайплайн аугментации изображений на основе библиотеки Albumentations, включающий 15 различных преобразований. Экспериментально показано, что применение аугментации позволяет получать из одного исходного изображения множество реалистичных вариаций, увеличивая объем обучающей выборки и повышая обобщающую способность нейросетевых моделей. Цель работы – изучение методов аугментации и их практическая реализация – достигнута.

ПРИЛОЖЕНИЕ А

В данном приложении приведен Python-скрипт, реализующий пайплайн аугментации изображений с использованием библиотеки Albumentations. Скрипт считывает исходные изображения из папки train, применяет к ним преобразования и сохраняет аугментированные версии в папку augmented, доводя количество изображений в каждом классе до заданного (target_count_per_class).

```
import os
import cv2
import albumentations as A
from tqdm import tqdm

# Настройки
input_dir = "train"
output_dir = "augmented"
target_count_per_class = 100
extensions = ('.jpg', '.jpeg', '.png')

transform = A.Compose([
    A.HorizontalFlip(p=0.5),
    A.VerticalFlip(p=0.3),
    A.Rotate(limit=30, p=0.7),
    A.Affine(scale=(0.8, 1.2), translate_percent=(-0.1, 0.1), rotate=(-15,
15), p=0.5),
    A.RandomBrightnessContrast(brightness_limit=0.2, contrast_limit=0.2,
p=0.6),
    A.GaussNoise(std_range=(0.039, 0.196), p=0.4),
    A.GaussianBlur(blur_limit=(3, 7), p=0.3),
    A.CoarseDropout(num_holes_range=(1, 8), hole_height_range=(8, 32),
hole_width_range=(8, 32), p=0.3),
    A.ToGray(p=0.2),
    A.HueSaturationValue(hue_shift_limit=20, sat_shift_limit=30,
val_shift_limit=20, p=0.5),
    A.OpticalDistortion(distort_limit=0.05, p=0.3),
    A.GridDistortion(num_steps=5, distort_limit=0.3, p=0.3),
    A.ElasticTransform(alpha=1, sigma=50, p=0.2),
    A.RandomGamma(gamma_limit=(80, 120), p=0.4),
    A.CLAHE(clip_limit=2.0, tile_grid_size=(8, 8), p=0.3),
])

def augment_image(image, save_path, num_augmented):
    for i in range(num_augmented):
        augmented = transform(image=image)['image']
        base, ext = os.path.splitext(save_path)
        new_path = f"{base}_aug_{i}{ext}"
        cv2.imwrite(new_path, augmented)

def main():
    for class_name in os.listdir(input_dir):
        class_path = os.path.join(input_dir, class_name)
        if not os.path.isdir(class_path):
            continue
        out_class_path = os.path.join(output_dir, class_name)
        os.makedirs(out_class_path, exist_ok=True)
        images = [f for f in os.listdir(class_path) if
f.lower().endswith(extensions)]
        current_count = len(images)
        if current_count >= target_count_per_class:
```

```
            for img_name in images:
                src = os.path.join(class_path, img_name)
                dst = os.path.join(out_class_path, img_name)
                cv2.imwrite(dst, cv2.imread(src))
            continue
        need = target_count_per_class - current_count
        augs_per_image = (need + current_count - 1) // current_count
        for img_name in tqdm(images, desc=f"Аугментация {class_name}"):
            img_path = os.path.join(class_path, img_name)
            image = cv2.imread(img_path)
            if image is None:
                continue
            out_original = os.path.join(out_class_path, img_name)
            cv2.imwrite(out_original, image)
            augment_image(image, out_original, augs_per_image)

if __name__ == "__main__":
    main()
```